

Session plan.md

Plan: Healthy One Gram Partner API Blueprint

Design a partner-facing Product Data API as a secure, read-only surface on top of the existing BogEcom backend, with strict data minimization, API-key auth, per-partner rate limits, webhook delivery for catalog events, OpenAPI-first docs, and a phased rollout that preserves current production APIs.

Steps

1. Define scope boundary and exposure policy (Phase 0)

- Publish explicit allowlist of fields partners can access (catalog, pricing, stock, nutrition, urls, optional shipping/discount summary).
- Publish explicit denylist of fields never exposed (customer/order/payment/supplier/cost/internal flags).
- Decide integration mode for v1: catalog sync + redirect-to-buy only (no partner order placement).

2. Create API surface contract first (Phase 1) (*blocks steps 3-7*)

- REST-first under /api/v1/partner with JSON-only responses.
- Standard response envelope, error schema, pagination schema, filtering/search conventions.
- Versioning policy: URL versioning for breaking changes, additive fields allowed in same version.
- Generate OpenAPI spec for partner subset and publish docs portal.

3. Build partner identity and auth model (Phase 2)

- Add Partner entity with status, scopes, rate plan, webhook secret, allowed origins, and key metadata.
- Implement API key auth using signed prefix + hashed secret at rest; support key rotation and soft-expiry.
- Optional JWT bearer mode for advanced partners (token minted from API key) if short-lived tokens are required.
- Enforce scope-based access per endpoint (catalog.read, inventory.read, price.read, webhook.read).

4. Build read models and endpoint handlers (Phase 3)

- Introduce partner-safe DTO mapper from Product model to external schema.
- Add endpoints for products list/detail, inventory list/detail, categories/tags, and metadata for sync cursors.
- Implement filtering (category, tag, brand, inStock), sorting, full-text search, cursor or page+limit pagination.
- Add ETag and Last-Modified support for efficient partner polling.

5. Add webhook subsystem for partner sync (Phase 4)

- Events: product.created, product.updated, stock.updated, price.updated.
- Delivery model: at-least-once with signed payloads, retries with exponential backoff, dead-letter queue.
- Webhook management APIs: register endpoint, rotate secret, pause/resume, replay window.
- Idempotency guidance using eventId + occurredAt + entityVersion.

6. Harden security and abuse controls (Phase 5)

- Per-partner rate limits (token bucket), burst + sustained quotas.
- WAF/CDN filtering, IP allowlist optional, anomaly alerts for scrape spikes.
- Response shaping to prevent leakage (server-side field whitelist only).
- Audit logs for key use, endpoint access, webhook delivery outcomes.

7. Performance and scale readiness (Phase 6)

- Redis caching for hot list endpoints and category/tag indexes.
- CDN cache for static image urls and product payload GET responses where allowed.
- Read-optimized queries with indexed filters (isActive, category, tags, updatedAt, stock state).
- Async change propagation to webhooks through queue workers.

8. Partner onboarding and operations (Phase 7)

- Admin flow: create partner, assign plan/scopes, generate key, share sandbox docs.
- Self-serve onboarding docs and Postman collection.
- Health/status endpoints and support playbook.
- Metering dashboard: calls, errors, latency, webhook success rate.

9. Future expansion path (Phase 8)

- Add partner order-intent/order-create APIs behind separate scopes and stricter compliance controls.

- Add attribution and affiliate commission tracking via partnerId/clickId/ref parameters.
- Add usage analytics and billing tiers.

****System architecture diagram (text)****

- Partner Client (web/app/marketplace)
- > API Gateway / CDN (TLS, WAF, rate-limits, caching)
- > Partner API Service (Express module in existing server or dedicated microservice)
- > Auth Layer (API key validation, scope checks, partner status)
- > Partner DTO Mapper (safe field projection)
- > Catalog Query Layer (Mongo read models, indexed queries)
- > Cache Layer (Redis)
- > Response Layer (ETag, pagination, errors)
- > MongoDB (products, categories, inventory)
- > Event Publisher (product/stock/price change events)
- > Webhook Queue Worker (retry, signature, dead-letter)
- > Partner Webhook Endpoint

****Recommended API endpoint structure****

- GET /api/v1/partner/health
- GET /api/v1/partner/products
- query: page, limit, q, category, tag, inStock, updatedSince, sort
- GET /api/v1/partner/products/{productId}
- GET /api/v1/partner/inventory
- query: sku, productId, inStock, updatedSince, page, limit
- GET /api/v1/partner/categories
- GET /api/v1/partner/tags
- GET /api/v1/partner/pricing
- query: productId, sku, currency
- GET /api/v1/partner/discounts (optional)
- GET /api/v1/partner/shipping (optional summary)
- POST /api/v1/partner/webhooks
- GET /api/v1/partner/webhooks
- PATCH /api/v1/partner/webhooks/{webhookId}
- POST /api/v1/partner/webhooks/{webhookId}/rotate-secret
- POST /api/v1/partner/webhooks/{webhookId}/replay

****Example request/response JSON****

1. Catalog list request

- Request:
- GET /api/v1/partner/products?page=1&limit=20&inStock=true&q=peanut
- Headers: Authorization: Bearer PARTNER_API_KEY

- Response:

```
{
  "success": true,
  "data": [
    {
      "id": "699dca97931ce0ec13a8f748",
      "sku": "HOG-PB-500G-CRUNCHY",
      "name": "Healthy One Gram Crunchy Peanut Butter",
      "description": "High-protein peanut butter with crunchy texture.",
      "shortDescription": "Protein-rich crunchy spread",
      "images": ["https://cdn.../pb-500g-1.jpg"],
      "productUrl": "https://www.healthystonegram.com/product/699dca97931ce0ec13a8f748",
      "category": {"id": "cat123", "name": "Peanut Butter"},
      "tags": ["high-protein", "no-added-sugar"],
      "price": {"amount": 399, "currency": "INR", "originalAmount": 499},
      "discount": {"type": "percentage", "value": 20},
      "stock": {"status": "in_stock", "availableQuantity": 142},
```

```

"nutrition": {
  "servingSize": "32g",
  "calories": 190,
  "protein": "8g",
  "fat": "16g",
  "carbs": "6g"
},
"shipping": {"freeShipping": true, "estimatedDispatchDays": 1},
"updatedAt": "2026-03-18T09:30:00.000Z"
}
],
"pagination": {"page": 1, "limit": 20, "total": 312, "totalPages": 16},
"meta": {"requestId": "req_abc123", "version": "v1"}
}

```

2. Error format

```

{
  "success": false,
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Rate limit exceeded. Retry after 60 seconds.",
    "details": null
  },
  "meta": {"requestId": "req_xyz789"}
}

```

3. Webhook payload example

```

{
  "eventId": "evt_01JQ...",
  "eventType": "stock.updated",
  "occurredAt": "2026-03-18T10:05:11.000Z",
  "data": {
    "productId": "699dca97931ce0ec13a8f748",
    "sku": "HOG-PB-500G-CRUNCHY",
    "stock": {"status": "in_stock", "availableQuantity": 120},
    "updatedAt": "2026-03-18T10:05:08.000Z"
  },
  "signature": "sha256=..."
}

```

****Database design (Mongo collections)****

- partners
- _id, name, status, contactEmail, allowedOrigins[], scopes[], ratePlanId, createdAt, updatedAt
- partner_api_keys
- _id, partnerId, keyPrefix, keyHash, lastUsedAt, expiresAt, revokedAt, createdBy
- partner_webhooks
- _id, partnerId, url, secretHash, eventTypes[], status, retryPolicy, createdAt, updatedAt
- partner_webhook_deliveries
- _id, webhookId, eventId, attempt, statusCode, responseTimeMs, nextRetryAt, payloadHash
- partner_usage_daily
- _id, partnerId, date, requests, errors4xx, errors5xx, avgLatencyMs, webhookSuccessRate

****Caching strategy****

- Redis:
- Product list cache keys by normalized query (short TTL: 30-120s)
- Product detail cache (TTL: 120-300s)
- Category/tag cache (TTL: 10-30m)
- CDN:
- Cache GET responses where partner-specific auth does not alter payload.

- Always cache product images via Cloudinary/CDN urls.
- Invalidation:
- On product/price/stock update, invalidate affected keys and enqueue webhook events.

****Tech stack recommendation****

- Preferred now: Node.js + Express (reuse existing server codebase, middleware, OpenAPI generation, rate limiter, auth patterns).
- Queue/events: Google Pub/Sub (already present) or BullMQ+Redis if you want tighter retry control in one stack.
- Data store: Existing MongoDB with new partner collections.
- Alternative (if greenfield): FastAPI + PostgreSQL for strict schemas, but migration cost is higher today.

****Relevant files to reuse in current repo****

- c:\BogEcom\server\index.js — route registration and middleware wiring pattern.
- c:\BogEcom\server\middlewares\auth.js — bearer token extraction/verification pattern.
- c:\BogEcom\server\middlewares\rateLimiter.js — baseline limit strategy to adapt per partner key.
- c:\BogEcom\server\models\product.model.js — product fields and stock/nutrition availability.
- c:\BogEcom\server\routes\product.route.js — filtering/search endpoint patterns.
- c:\BogEcom\server\routes\webhook.route.js — webhook signature verification template.
- c:\BogEcom\OPENAPI_SPEC.generated.json — existing contract generation baseline.
- c:\BogEcom\scripts\generate-openapi-from-source.cjs — OpenAPI automation pipeline.
- c:\BogEcom\dispatch.yaml and c:\BogEcom\server\app.yaml — deployment and routing baseline.

****Deployment recommendation****

- Domain: api.healthyonegram.com
- Route segmentation:
- /api/v1/partner/* for partner API
- Keep internal/admin APIs unchanged
- Hosting:
- Short term: existing Google App Engine service with isolated route/middleware.
- Scale-out path: dedicated partner-api service behind same domain and gateway routing.
- Ops essentials:
- Centralized logs, request IDs, error alerts, SLO dashboards, webhook DLQ visibility.

****Partner onboarding flow****

1. Partner submits registration form (company, use case, expected QPS, webhook url).
2. Admin approves and creates partner profile + scopes + rate plan.
3. System generates API key once (display once), stores hash only.
4. Partner tests in sandbox with docs and sample data.
5. Production activation with monitoring and webhook verification handshake.

****Verification****

1. Contract tests for all /api/v1/partner endpoints against OpenAPI examples.
2. Security tests: denylist fields never appear in payload snapshots.
3. Load tests for list endpoints with cache disabled/enabled.
4. Webhook tests: signature verification, retries, replay, idempotency.
5. Partner UAT checklist: initial sync, delta sync, out-of-stock update latency.

****Decisions****

- Included in v1:
- Read-only catalog/pricing/inventory API
- API key auth + scopes + per-partner limits
- Webhooks for product/price/stock/new product events
- OpenAPI docs + partner onboarding workflow
- Excluded from v1:
- Customer/order/payment APIs
- Supplier/cost/internal admin logic
- Partner order placement (planned v2)

****Further Considerations****

1. Auth mode recommendation

- Option A: API key only (fastest rollout).
- Option B: API key + short-lived JWT exchange (better enterprise posture).
- Recommendation: Start with Option A and add B for large partners.

2. Versioning strategy

- Option A: URI-only (/api/v1).
- Option B: URI + sunset headers per endpoint.
- Recommendation: Option B for controlled deprecations.

3. Webhook delivery guarantee

- Option A: At-least-once with retries (simple/reliable).
- Option B: Exactly-once semantics (high complexity).
- Recommendation: Option A + idempotency keys for v1.